

How VoxPin works

A tiny pin records your voice. A program on the Mac figures out what you asked. Then Google, the calendar, or a parent's phone finishes the job, and the pin talks back.

■ On the pin
 ■ On the Mac
 ■ Outside services
 ■ Help / alarm

1 • THE PIN

ESP32-S3 board

- Microphone and speaker
- Small color screen
- Talk button, plus button, power button
- Motion sensor and battery
- Wi-Fi and Bluetooth

2 • THE MAC HELPER

Python program

- Listens at `http://...:8765`
- Shouts its address every second on UDP port 8766
- Turns speech into words
- Saves notes, reminders, and translations
- Sends help texts

3 • OUTSIDE

Google and the web

- Speech-to-text
- Google Doc for notes
- Google Calendar for reminders
- Weather, maps, and answers
- iMessage to a parent

You talk

Hold the talk button. The pin records up to 20 seconds.

Pin sends audio

Raw sound goes to the Mac at `/note`.

Mac picks a job

Notes, translate, remind, ask, timer, weather, or help.

Pin answers

It shows a short status and plays the spoken reply.

Jobs that run at the same time

After `setup()` in `voxpinned.ino`, the pin starts these loops. The Mac starts one web server.

On the pin

`firmware/lcd-0.85`

1. `voice_note_task` – wait for the talk button
2. `side_buttons_task` – wait for the plus (help) button
3. `reminder_task` – check the calendar
4. `idle_task` – sleep, wake, or power off
5. `home_update_task` – clock, battery, next event
6. `lvgl_task` – draw the screen

On the Mac

`backend/voice_notes/app.py`

1. `main()` – start the website on port 8765
2. `start_helper_beacon()` – broadcast “VOXPIN ip port”
3. `note()` – handle a voice clip
4. `sos_alert()` – handle the help button
5. `reminder_due()` and `announce_reminder()` – alarm audio
6. Companion pages – notes, language, calendar

Print this packet in color if you can. The boxes still read in black and white.

What happens when the pin turns on

`setup()` in `voxpinned.ino` runs once. `loop()` is empty because the other jobs keep running by themselves.

setup()

`voxpinned.ino`

1. `power_hold_on()` holds the battery switch on so the board stays powered.
2. `power_init()` measures battery voltage on GPIO1 and checks if it is charging.
3. `wifi_connect_begin()` joins the saved Wi-Fi (up to 45 seconds).
4. If Wi-Fi works, `home_ui_sync_time_from_ntp()` sets the clock from the internet.
5. `ble_companion_begin()` starts Bluetooth so a phone can read the name and language.
6. `home_ui_begin()` turns on the screen and draws the home page.
7. `voice_note_init()` reserves microphone memory and starts the speaker chip.
8. `voice_note_start()`, `side_buttons_start()`, `reminder_alert_start()`, and `idle_start()` each start their own loop.

Finding the Mac

`wifi_connect.cpp` · `backend_discover()`

1. Listen on UDP port 8766 for a message that starts with **VOXPIN**, then an IP and a port.
2. Ask that address for `/health`. If it answers, remember it.
3. If the beacon is missing, try saved addresses and the Wi-Fi gateway on port 8765.
4. `backend_fallback_cloud()` tries the cloud helper only if the Mac on the same Wi-Fi is not there.

start_helper_beacon()

`app.py`

1. `lan_ipv4s()` lists the Mac's Wi-Fi addresses.
2. Every second, send **VOXPIN** `<ip> 8765` to the local network.
3. The pin uses that message so you do not type an IP address.

main()

`app.py`

1. `--login` opens a browser so Google Calendar and Docs can be used.
2. Otherwise print the website address and start Flask on port 8765.
3. Notes and translations go to two different Google Docs.

Inside the talk button

`voice_note_task()` loops forever, waiting for the BOOT button.

voice_note_task()

voice_note.cpp

1. Wait until the talk button is pressed. Check every 20 ms.
2. `idle_touch()` marks the pin as busy. If it was sleeping, `idle_wake_sync()` turns Wi-Fi and the bright screen back on.
3. Show “Recording” and read the microphone in 4096-byte chunks, 16,000 samples per second, one channel, 16-bit.
4. After you let go, keep a little more audio, then stop on quiet sound or after 20 seconds.
5. `trim_pcm16()` cuts silence off the ends. If the clip is shorter than 1/4 second, show “Too short” and wait again.
6. Call `handle_clip()`, which turns Wi-Fi power-save off so the upload is faster.

handle_clip_inner()

voice_note.cpp

1. If Wi-Fi is down, wait up to 10 seconds. Show “No WiFi” if it never joins.
2. POST the sound bytes to `/note` with sample rate, channels, and bits in the headers.
3. If the Mac does not answer, try the cloud helper once.
4. HTTP 204 means the Mac heard nothing useful. The screen says “Didn't hear”.
5. HTTP 200 means a short result with no audio, such as “Saved”, “Help sent”, or “Timer set”.
6. HTTP 201 means the body is spoken audio. `play_pcm()` plays it on the speaker, then the status clears.

What the Mac does first inside note()

app.py

1. Read the sound bytes and the sample-rate headers.
2. `pcm_to_wav()` wraps the raw sound in a WAV file.
3. `transcribe()` sends that sound to Google speech-to-text in English, then `fix_words()` cleans the text.
4. `parse_command()` looks at the first words and chooses the job. The next page is that choice.

Inside `parse_command()`

The function reads the sentence from the start. The first matching phrase wins. If you do not say a command, the words are saved as a note.

note

"take notes ..." "write this down"

1. `store.add_recording("note")` saves the text on the Mac.
2. `append_to_doc_later()` adds "Note: ..." to the Google Doc.
3. The pin speaks "Notes added." and shows "Saved".

translate

"translate this ..."

1. Read the language chosen on the website (default Spanish).
2. `translate_text()` translates the phrase.
3. `speak_translated_pcm()` makes speech with the Mac `say` voice.
4. Save the pair in the translations Google Doc. The pin plays it.

remind

"remind me to ..."

1. If Calendar is not signed in, the pin says to sign in on the Mac.
2. `parse_reminder()` finds the title, the time, and how long it lasts.
3. `create_calendar_event()` writes it to Google Calendar.
4. The pin speaks the title and time, and shows "Reminded".

ask

"what's ..." "tell me ..." "who is ..."

1. `answer_question()` checks the kind of question.
2. Time questions use the Mac clock. Note questions read the latest notes.
3. Calendar questions use `speak_reminders()`.
4. Other questions try DuckDuckGo, then Wikipedia. The pin speaks the answer.

timer

"set a timer ..." "10 minutes"

1. `family.parse_timer()` turns words like "ten minutes" into a number.
2. A calendar event is created for that many minutes from now.
3. The screen shows "Timer 10m". No long speech is sent back.

weather

"what's the weather"

1. `family.geolocate()` finds a place from the network.
2. `weather_summary()` asks Open-Meteo for temperature and rain.
3. The pin speaks a short line, such as "It's 68 degrees."

location or sos

"where am I" · "help me"

1. Both call `handle_parent_ping()`.
2. Location shows "Sent loc". SOS tries to text a parent. See the help page.

Nothing matched

empty or unclear audio

1. If there is no transcript, `note()` returns HTTP 204.
2. The pin shows "Didn't hear" and does not save anything.

Inside the plus button and reminders

The plus button does not record your voice. It sends nearby Wi-Fi signals so the Mac can guess the place.

side_buttons_task()

side_buttons.cpp

1. Wait until the plus button is pressed, then wait until it is released.
2. Wake the pin if it was sleeping. Show "Sending help".
3. `scan_wifi_json()` lists up to 8 nearby Wi-Fi routers and how strong they are.
4. `post_event("/sos")` POSTs that list to the Mac.
5. The screen shows "Help sent", "Sent already", "Msg failed", or "No iMessage".

sos_alert() → handle_parent_ping("sos")

app.py and family.py

1. `geolocate()` sends the Wi-Fi list to a location service. If that fails, it uses the public IP address, then a city name.
2. `reverse_geocode()` turns the latitude and longitude into a neighborhood and city, plus a Google Maps link.
3. `store.add_alert()` keeps the alert on the Mac.
4. `notify_parent()` shows a Mac notification. For SOS it also runs Messages and sends "Sending from Voxpin, please pick me NOW!" plus the map link.
5. A second SOS within 5 seconds is skipped so the phone is not spammed.

reminder_task()

reminder_alert.cpp

1. While you are using the pin, check every 20 seconds. While sleeping, wait longer, up to 10 minutes.
2. Skip the check if the pin is already recording or speaking.
3. `fetch_due()` asks the Mac `/reminder-due`. The Mac looks about 10 minutes ahead on Google Calendar.
4. If an event is within 12 minutes, wake the pin and keep Wi-Fi on.
5. If this exact event was already announced, do nothing.
6. Otherwise show "Reminder", `play_buzz()`, then `speak_due_reminder()` downloads speech from `/announce-reminder` and plays it.

Inside the screen, sleep, and Bluetooth

These functions keep the pin usable without you holding a button.

home_update_task()

home_ui.cpp

1. Once a second, read the clock and `power_battery_percent()`.
2. A charging battery gets a plus sign, like 80%+.
3. `refresh_event_label()` asks /next-event at most every 30 seconds.
4. The screen shows time, date, battery, and the next calendar title.
5. `home_ui_set_status()` writes words like "Recording" over that screen.

lvgl_task()

home_ui.cpp

1. This is the drawing loop for the small LCD.
2. It clears a status message when its timer runs out.
3. While sleeping, it draws less often (every 200 ms instead of 20 ms) to save power.

idle_task()

idle.cpp

1. If nobody touches the pin for 5 minutes, `enter_sleep()` runs.
2. Sleep dims the backlight, turns Wi-Fi and Bluetooth ads off, and slows the chip to 80 MHz. The clock stays on the screen.
3. `imu_moved()` wakes it if you pick it up.
4. A short press of power toggles sleep. Holding power for 2 seconds calls `power_off()`.

leave_sleep()

idle.cpp

1. Speed the chip back up to 240 MHz.
2. Brighten the screen, turn the speaker amp on, and advertise Bluetooth again.
3. `wifi_wake_start()` rejoins Wi-Fi in the background.
4. The screen says "Waking" for a moment.

ble_companion_begin()

ble_companion.cpp

1. The pin advertises the name "VoxPin".
2. A phone can read or write a small profile: name and language.
3. The profile is stored on the pin with Preferences, so it stays after restart.

power_battery_percent()

power.cpp

1. Read the battery voltage with the analog pin.
2. 4.15 volts is about 100%. 3.20 volts is about 0%.
3. `power_is_charging()` checks the charger pin so the screen can show a plus.

Companion website

open <http://127.0.0.1:8765/>

1. **Notes** shows text the pin saved. It does not store the audio clips.
2. **Language** is the language `translate` speaks. The pin stores a copy over Bluetooth.
3. **Calendar** shows Google Calendar events. Signing in with Google lets the site read the Doc and the calendar without the pin.
4. Voice from the pin can still write into that same Doc through the Mac helper.